SIMVA: cambios, pruebas, pendientes y preguntas abiertas

Actualizado el 24 oct 2026. Este documento resume lo que cambió en el código desde la revisión del 19 oct, cómo probarlo, qué falta y qué decisiones siguen sin respuesta. El README tiene la descripción general del proyecto.

1. Qué cambió

Decisiones ya tomadas (no se vuelven a discutir)

- **Listado final de hardware confirmado:** ESP32, Raspberry Pi 4 (4 GB) con microSD, SCD41, MAX30102, ventilador Steren 12 V con módulo D4184, módulo vibratorio 5 V, LED, buzzer **activo**, fuentes de 12 V y 5 V, protoboard, jumpers, resistencias y diodos 1N5819.
- **El AM2302 sale del proyecto.** El SCD41 ya mide temperatura y humedad. Se borró `esp32/leer_am2302.ino`.
- **El relé no se usa en el firmware.** El D4184 conmuta y regula el ventilador por PWM (ver pregunta 1).
- **Ventilador proporcional al CO_2. Vibración** apagada en Normal, 50 % en Alerta, 100 % en Emergencia.
- **El ESP32 ejecuta la lógica de control:** aplica los niveles que manda el servidor y, si no hay respuesta, decide solo con el CO_2 (respaldo local).

Código (commits desde el 19 oct)

Commit	Archivos	Cambio
`2762fc4`	`raspberry/logica.py`	Sin argumento mutable por defecto.
`3acc59e`	`raspberry/servidor.py`	`/lectura` responde `estado` y niveles; `/estado` incluye `estado`; nuevo `POST /operador`.
`635f307`, `72f8245`	`interfaz/`	Tarjeta de Estado, operador enviado al servidor, aviso de datos simulados, eje Y flexible, manifest sin ícono inexistente.
`267da0c`	`esp32/main_esp32.ino`	Control de actuadores con la respuesta del servidor; no envía datos inventados si falla un sensor.
`c7560db`	`raspberry/logica.py`, `servidor.py`	Ventilador proporcional al CO_2 y vibración por estado; la API devuelve `ventilador_pct` y `vibracion_pct`.
`081efca`	`interfaz/index.html`	Porcentaje del ventilador, tarjeta de vibración, `--` cuando el pulso es 0.
`61fae72`	`esp32/`	Firmware nuevo: SCD41 y MAX30102 por I^2C, PWM del ventilador y del vibrador, LED, buzzer y respaldo local.
(este commit)	`docs/`, `.gitignore`, READMEs	Documentación actualizada (sección 5) y `raspberry/ia/models/` ignorado.

Reglas de salida (`raspberry/logica.py`)

Condición	Estado	Ventilador	Vibración	LED	Buzzer
CO_2 < 1000 sin riesgo	Normal	0 %	0 %	apagado	apagado
IA anticipa el umbral (< 5 min)	Alerta	30 %	50 %	encendido	apagado
CO_2 ≥ 1000 ppm	Alerta	40 % a 1000 ppm, sube lineal hasta 100 % a 2000 ppm	50 %	encendido	apagado
CO_2 > 2000, o CO_2 > 1000 con pulso > 25 % sobre 70 bpm	Emergencia	100 %	100 %	encendido	encendido

El ESP32 usa las mismas reglas con el CO_2 local (`controlLocal()`) si no hay WiFi o el servidor tarda más de 3 s.

2. Conexiones propuestas (provisionales, confirmar con Ingrid)

Elemento	Conexión	Notas
SCD41	3.3 V, GND, SDA = GPIO 21, SCL = GPIO 22	Dirección I^2C 0x62.
MAX30102	3.3 V, GND, mismo SDA/SCL	Dirección 0x57. Comparte bus con el SCD41.
Ventilador (D4184)	Entrada PWM ← GPIO 26. Fuente de 12 V en el módulo; ventilador en su salida	Diodo 1N5819 en antiparalelo con el ventilador (cátodo al +12 V). Revisar la serigrafía del módulo.
Módulo vibratorio	Señal ← GPIO 27. Alimentación 5 V de la fuente de 5 V	Ver pregunta 3.
LED	GPIO 25 → resistencia (220 a 330 Ω) → LED → GND	
Buzzer activo	GPIO 33 → GND	Si consume más de 12 mA, usar un transistor o módulo con driver.
GND común	GND del ESP32, de la fuente de 12 V y de la fuente de 5 V unidos	Obligatorio antes de energizar (PRR 2).

Los pines se cambian en las constantes del inicio de `esp32/main_esp32.ino`.

3. Cómo probar

Paso 0. Preparar

```
cd raspberry
pip install -r requirements.txt
python ia/train.py        # genera ia/models/co2_model.joblib (opcional: sin él solo se usa la regla de pendiente)
```

Paso 1. Servidor y lógica (sin hardware)

1. En una terminal: `python servidor.py` (puerto 8080). Si existe `lecturas.db` de pruebas anteriores, bórrala antes para empezar limpio.
2. Guarda este script como `prueba_api.py` (fuera del repo o en cualquier carpeta) y ejecútalo en otra terminal con `python prueba_api.py`:

```
import json, urllib.request

URL = "http://localhost:8080"

def post(ruta, datos):
    req = urllib.request.Request(URL + ruta, json.dumps(datos).encode(),
                                 {"Content-Type": "application/json"})
    return json.load(urllib.request.urlopen(req))

print(post("/operador", {"operador": "ivan"}))
for co2 in (500, 1000, 1500, 2100):
    print(co2, post("/lectura", {"co2": co2, "temp": 24, "hum": 50, "pulso": 0}))
print(json.load(urllib.request.urlopen(URL + "/estado")))
```

3. **Resultado esperado** (verificado el 24 oct, con y sin modelo entrenado):

CO_2	estado	ventilador_pct	vibracion_pct
500	Normal	0	0
1000	Alerta	40	50
1500	Alerta	70	50
2100	Emergencia	100	100

Las lecturas se acumulan en la base: la IA usa las últimas 10, así que una subida rápida (por ejemplo 500 a 900) puede dar Alerta anticipada con ventilador al 30 %. Es normal.

4. Pulso alto: envía `"co2": 1200, "pulso": 100` y debe dar **Emergencia** (pulso > 25 % sobre 70 bpm con CO_2 > 1000).
5. Entradas inválidas: `POST /lectura` con `{"co2": "abc"}` debe responder 400.

Paso 2. Dashboard

1. Con el servidor activo abre `http://localhost:8080`.
2. Tras el paso 1 debe mostrar Estado, CO_2, ventilador con su porcentaje y vibración. Sin lecturas muestra "Sin datos registrados".
3. Detén el servidor: en 3 s debe aparecer **"⚠ Datos simulados (sin servidor)"**.
4. Cambia el operador en el selector y envía una lectura sin campo `operador`: `GET /ultimas` no lo muestra, pero la base guarda el operador elegido (`sqlite3 lecturas.db "select operador from lecturas order by id desc limit 1"`).

Paso 3. Firmware del ESP32 (por etapas)

Preparación en Arduino IDE:

- Instalar el soporte de placas *esp32 by Espressif* y elegir *ESP32 Dev Module*. El código acepta el core 2.x y el 3.x.
- Instalar las librerías *Sensirion I2C SCD4x* y *SparkFun MAX3010x Pulse and Proximity Sensor Library*.
- En `main_esp32.ino` poner `ssid`, `password` y la IP de la Raspberry en `serverUrl` (conviene reservar una IP fija para la Raspberry).

- **El firmware aún no se ha compilado.** Si falla al compilar, probablemente es el nombre de algún método de las librerías según su versión: avisa con el mensaje de error.

Etapas (monitor serial a 115200):

1. **Solo SCD41 + WiFi.** Debe imprimir cada 5 s `Enviando JSON: {...}` y `Respuesta Servidor: 200 {...}`. Al soplar sobre el sensor el CO_2 debe subir y aparecer en el dashboard.
2. **Agregar MAX30102.** Con el dedo sobre el sensor, `pulso` debe estabilizarse en 60 a 100 bpm en unos segundos; sin dedo debe enviar `0`. (Los latidos se detectan solo si el loop corre continuo.)
3. **Salidas sin carga.** Con un multímetro o LED en cada pin: GPIO 26 y 27 deben variar su tensión media con el CO_2 (PWM), GPIO 25 se enciende en Alerta/Emergencia y GPIO 33 solo en Emergencia. Se puede forzar enviando lecturas desde `prueba_api.py` y mirando la respuesta en el serial.
4. **Con carga** (solo después de unir GND y poner el diodo): ventilador por el D4184 y módulo vibratorio. Verificar que el ventilador arranque con 40 % de ciclo; si no gira, subir ese mínimo en `logica.py` (y en `controlLocal()`).
5. **Respaldo local.** Apaga el servidor (o el WiFi): en el siguiente ciclo debe imprimir `Error enviando POST` o `Wi-Fi desconectado` y seguir accionando los actuadores según el CO_2.

Paso 4. Pruebas del Project Charter

Criterio	Cómo probarlo
Lecturas cada ≤ 5 s	Dejar 10 min y revisar `lecturas.db`: `select fecha from lecturas` debe mostrar saltos de ~5 s sin huecos.
Ventilador y LED en ≤ 5 s tras superar 1000 ppm	Exhalar en una caja cerrada con el sensor; cronometrar desde que el serial muestra más de 1000 ppm hasta que cambian ventilador y LED.
Anomalía provocada, registrada y visible	Provocar una subida rápida de CO_2; hoy el dashboard muestra el diagnóstico IA, pero **no hay registro histórico de anomalías** (pendiente, sección 4).
Diferencia ≤ 15 % frente al monitor de referencia	Medir en paralelo con el monitor comercial y documentar la comparación.

4. Lo que falta

Software

- Compilar y depurar el firmware del ESP32 (nunca se ha compilado).
- Registro y visualización de anomalías y un histórico más allá de los últimos 15 puntos de la gráfica (criterios del Charter).
- Reentrenar el modelo con datos reales; hoy usa datos sintéticos.
- Servir Chart.js localmente si la Raspberry no tendrá internet, y crear `interfaz/icon.png` si se quiere instalar como app.
- Fijar versiones en `requirements.txt` (riesgo 15 del plan de riesgos).

Hardware

- Armar y revisar el circuito de potencia (PRR 2): revisarlo con un compañero antes de energizar, GND común y diodos.
- Calibrar el CO_2 del SCD41 y comparar con el monitor de referencia.

Documentos que no pude editar o requieren decisión del equipo

- `docs/01_Project_Charter_WBS.pdf` es un PDF y no lo puedo modificar. Hay que actualizar a mano su lista de equipo (agregar D4184, módulo vibratorio, fuente de 5 V, diodos; quitar el AM2302 si aparece) y revisar el presupuesto de ~$1,100 MXN.
- `docs/02_Plan_Gestion_Riesgos.xlsx`, PRR 2: la acción "*Usar módulo relé con optoacoplador*" aparece como Completada y ya no corresponde al listado final (pregunta 1). El PRR 1 todavía menciona el AM2302 en el sensor de respaldo; si el SCD41 ya llegó, ese plan B se puede cerrar.
- Falta el **diagrama de conexiones** que pide el Charter (la tabla de la sección 2 es el borrador).
- Los riesgos 13 y PRR 6 hablan de una API externa de IA que el código no usa (pregunta 8).

5. Documentación ya actualizada

- `README.md`: listado final de hardware, arquitectura, API con niveles de ventilador y vibración, reglas, instalación de librerías y comentarios para el equipo.
- `docs/02_Plan_Gestion_Riesgos.xlsx`: riesgo 3 y PRR 2 ahora hablan de 12 V, 5 V y D4184 (tierras comunes de las dos fuentes, diodos 1N5819); nombre de Leonardo corregido a *Nájera*.
- `docs/05_Plan_Interesados.docx`: nombre de Leonardo corregido a *Nájera*.
- `docs/README.md` y `raspberry/README.md`: contenido real de las carpetas.

6. Preguntas que siguen sin respuesta

Las que no tenían respuesta el 24 oct, con la decisión provisional que tomé para no bloquear el trabajo:

1. **¿El relé sigue físicamente en el circuito?** Preguntar a Ingrid. Provisional: no se usa; el D4184 hace el trabajo. Si se queda como corte de seguridad, definir su pin y regla.
2. **¿Alguien necesita el AM2302?** Provisional: eliminado (SCD41 mide temperatura y humedad). Confirmar que no se usa como segunda referencia.
3. **¿El módulo vibratorio acepta PWM directo por su pin de señal** o necesita un transistor? Revisar su ficha.
4. **¿El ventilador Steren soporta PWM?** Depende de si es de 2 hilos (el D4184 corta la alimentación) o de 3/4 hilos (tiene entrada PWM propia). Confirmar frecuencia y mínimo de arranque (hoy 5 kHz y 40 %).
5. **¿Los pines propuestos coinciden con el circuito armado?** (ventilador 26, vibrador 27, LED 25, buzzer 33, I^2C 21/22.)
6. **¿La curva del ventilador es la deseada?** Dijiste "proporcional al CO_2"; los puntos (40 % a 1000 ppm, 100 % a 2000 ppm) y el 30 % anticipado son mi propuesta.
7. **Operador global:** el dashboard fija un solo operador para todas las lecturas (gana el último que cambió el selector). ¿Es suficiente para la demo?
8. **API externa de IA:** el plan de riesgos asume un servicio tipo ChatGPT, pero el código es local (scikit-learn). ¿Se va a usar? Si no, actualizar riesgo 13 y PRR 6.
9. **¿Qué cuenta como "anomalía"** para el criterio del Charter y dónde se registra/muestra?
10. **Raspberry en la demo:** ¿tendrá internet (para Chart.js por CDN)? ¿IP fija para el ESP32?
11. **Liderazgo en el Charter:** el documento nombra a Ingrid como "líder de proyecto, hardware…" y a Iván como líder. ¿Cuál es la correcta? (El README dice Iván.)
12. **Portada del Charter:** lista 4 integrantes; falta Mauricio.